

Amazon IEEE Deep dives

scalability

we address our scalability issue by

by separating our customer side from our warehouse side so our services can scale independently using Auto scale

and also by this we can optimize different scalability problems in our system

orders : we need to handle massive Reading thought put during hot sales , seasons sales etc so we added a Read-Replica to our postgres DB and we add a CDN caching so we can cache our responses but this is limited to our system scale if we have too many search queries the CDN might have alot of misses during search so it will be pointless

search : our Customer can search products by descriptions , Tags , name , etc so we added an elastic search cluster to our system which can perform well under scale and have an inside cache for search operations and we configure it to have a Real time updates with our postgres DB using CDC (change Data capture using pgsync)

robots : our robots can handle massive order scale each order once confirmed is sent as an event to Kafka topic ("paid orders") and a consumer on the other side robot-orchestrator is responsible of assigning and managing robots by Decoupling the producer the order event from our robots our system can handle the surge in order by just cosume the events from kafka at a Rate or cosuming multiple orders and the orchestrator will manage the robot to make the least travel possible thourght an optimization algorithm

we also can handle multiple robots in real time they are being sent to a socket server that can easily handle thousands of concurrent tcp connections, these raw data is passed into kafka stream

and then aggregated by FLINK service and make a fleet object with each robot and status and location etc.

each fleet is passed as an event that tasks service subscribe to to check for available fleets and their status, register fleet status in redis for fast access with LRU

and tasks service also send notifications on fleet status changes to operators

Notification : for notifications to handle millions of users and hot sales, we push a notification event to kafka message queue and multiple competing workers they first search the redis cache for device id if not found it update the cache and deliver the notification to Apple push notification or firebase cloud nessages, if messages fail they retry and if that fails too its sent to dead letter queue so that a worker can reread them sometime else

Reliability

our orders are idempotent (use_id + product_id []) so we will not process so our order service will not process the same order twice

robot failures our system uses web socket connection for its bidirectional and low latency and our web socket service stream Real time data that its gets from the robots to Kafka topic (robot data)

so if a robot doesnt report in the next 15 s we send a notification to the operators using the Task-service with the last location of the robot that we had in our kafka stream

we also wont process the same order twice to the robots once our robots are assigned to Tasks the Task service write a fast redis key-value with it

```
{
key Task_id
value {
order_id ,
robot_id
status "picked or unpicked"
}
}
```

with a TTL for 15 minutes

so we can know before assigning any robot to the Task our system check the redis cache to see if any robot are already assigned to this task or not if the robot didnt change its status to pick in 15 minutes the TTL will remove the key and a robots flagging service will append the Task back to the Task service to retry

if the robot picked the task successfully the Task service will change the task_id in redis to status :picked and notify the operators and customer

Availability

our system are availble during hot seasons and sales by autoScaling and decoupling the heavy work load through kafka topics

so our system are available

Latency

Query for our Customer must be <500Ms so our system prioritize this by caching , readReplica , Search caching through elastic search

our robots should sync Data in Real time so we user websockets to make a bidirectional so we could push an order to our robot or get real time updates with low latency and a priodic health check to our robots

consistency

to make sure that no inventory item is ordered twice when there is only one left or a race condition, we first call order service to create an order and decrease inventory by one and add it to db as pending with ttl 15m, then wait for payment in case of payment success status is successful' if not add inventory count by 1 again and make status as failed or if ttl expired

for availability to handle multiple robots in real time they are being sent to a socket server that can easily handle thousands of concurrent tcp connections, these raw data is passed into kafka stream

and then aggregated by FLINK service and make a fleet object with each robot and status and location etc.

each fleet is passed as an event that tasks service subscribe to to check for available fleets and their status, register fleet status in redis for fast access with LRU and tasks service also send notifications on fleet status changes to operators

fault tolerance

we add fault tolerance by using stream driven architecture to decouple services and enable retries for failed messages with a retention window

for availability we use websockets for real time and kafka message queue which is known for high performance unlike rabbitmq or other message queues

to make sure the system isn't down on hot sales, orders service can scale horizontally since all instances listen

to message queues events, notifications worker can auto scale when needed and can handle millions of notifications

for apache flink we use sliding window for each robot event so that we can aggregate the fleet in a given time window and in near real time

flagging service check if robot's job is inactive for too long or no received events with the robot id for a time window and flag it to operator